

CMP180 WLAN EVM 自動化量測系統

CMP180 WLAN TX EVM 自動化量測與分析系統 — 取代手動 CMsquares GUI 操作

SPEC Version 0.1.0

Status: Draft / Hardware Discovery Required

Phase 2 — Connection Framework

Python 3.11+

自動化 Rohde & Schwarz **CMP180** Radio Communication Tester 的 WLAN TX EVM 量測流程，取代目前透過 CMsquares GUI 手動操作、容易出錯又難以重現的測試方式。完整規格請見專案內的 [SPEC.MD](#)（中文，含背景、資料模型、SCPI 抽象層設計、驗收標準等完整細節）；本文件只涵蓋「如何安裝、執行、開發」。

目前狀態

專案目前處於 **Phase 0 / Phase 2** 階段：

- 尚未完成 CMP180 SCPI Discovery，`configs/scpi_command_map.yaml` 內除了標準 IEEE-488.2 共通指令（`*IDN?` `*OPT?` `*CLS` `*OPC?` `SYST:ERR?`）之外，所有 CMP180 專屬指令（Generator、WLAN TX Measurement、EVM/Power 讀值）都刻意留白（`null`）。
- 尚未有真正的量測 workflow（single measurement / power sweep / frequency sweep），目前只有：設定檔載入與驗證、儀器連線骨架（含 Mock 模式）、CLI、Tkinter GUI（設定與連線頁面）。
- `RF1.1` 是否為 Generator 尚未確認；只有 `RF1.5`（Analyzer）已確認。在完成硬體 **Discovery** 前，程式不會、也不應該真的開啟 RF。

安裝

需要 Python 3.11 以上版本。

```
python -m venv .venv
.venv\Scripts\activate
pip install -e ".[dev]"
```

若要用真實儀器連線（需要 pyVISA + VISA runtime），額外安裝：

```
pip install -e ".[hardware]"
```

使用方式

命令列 (CLI)

```
# 驗證設定檔（自動判斷是 instrument 還是 wlan baseline 設定）
python -m cmp180_evm validate-config configs/wlan_baseline.example.yaml

# 印出計畫執行的步驟，不會送出任何真實 SCPI 指令
python -m cmp180_evm dry-run --config configs/wlan_baseline.example.yaml

# 測試連線；沒有硬體時用 --mock
python -m cmp180_evm test-connection --mock
python -m cmp180_evm test-connection --instrument-config
configs/instrument.example.yaml
```

圖形介面 (GUI，設定與連線)

```
python -m cmp180_evm.gui
```

也提供打包好的單一執行檔 `CMP180-EVM-GUI.exe`（Windows，PyInstaller `--onefile`），直接雙擊即可開啟，不需另外安裝 Python。執行檔需要與 `configs/` 資料夾放在同一層目錄，才能找到預設的範例設定檔（也可以用 GUI 的「Browse...」自行選擇其他路徑）。

目前 GUI 涵蓋：載入 / 瀏覽設定檔、驗證設定檔、Dry Run 預覽、測試連線（可切換 Mock / 真實硬體）。之後量測 workflow（single measurement、power sweep、frequency sweep）完成後，GUI 會加入對應頁面——GUI 與 CLI 共用同一套 `cmp180_evm/actions.py`，不會各自維護一份邏輯。

打包成執行檔

在已安裝 `pyinstaller` 的環境下 (`pip install -e "[build]"`) , 於專案根目錄執行 :

```
python -m PyInstaller --noconfirm --onefile --windowed ^  
--name CMP180-EVM-GUI --paths src src/cmp180_evm/gui/__main__.py
```

產出的執行檔位於 `dist/CMP180-EVM-GUI.exe` 。發布時請把 `configs/` 資料夾複製到 `dist/` 內、與 `exe` 放在同一層, 讓使用者可以直接編輯 `YAML` 設定檔, 不需重新打包。

測試

```
pytest -q
```

硬體相關測試會標記 `@pytest.mark.hardware` , 不會在一般 CI / 本機測試中自動執行 (需要真實連上 `CMP180` 才有意義) 。

專案結構

```
src/cmp180_evm/  
├ cli.py           # argparse CLI 入口  
├ actions.py       # CLI 與 GUI 共用的高階操作 (validate/dry-run/test-connection)  
├ gui/             # Tkinter GUI  
├ config/          # YAML 載入、pydantic typed models、跨欄位驗證 (band/frequency 等)  
├ instrument/      # InstrumentSession protocol、真實 CMP180 session (pyvisa)、MockCmp180  
├ scpi/            # SCPI 指令唯一集中管理處: common 常數 + ScpiCommandRegistry  
└ utils/           # exceptions、logging  
  
configs/  
├ instrument.example.yaml    # 儀器連線 + RF routing  
├ wlan_baseline.example.yaml # 訊號/量測參數  
└ scpi_command_map.yaml     # CMP180 指令對照表 (未確認指令保持 null)  
  
tests/unit/          # pytest 單元測試
```

核心設計原則

這些規則來自 `SPEC.MD`，是本專案最容易被 AI 或新貢獻者不小心違反的地方，開發前請先讀過：

- 1 不可自行編造 **CMP180 SCPI 指令**。只能使用 `configs/scpi_command_map.yaml` 中已確認的指令；缺指令時要丟出 `ScpiCommandNotConfiguredError`，不能用其他 R&S 儀器（如 FSW85/K70）的指令代替猜測。
- 2 **RF ON 前必須通過安全檢查**（generator/analyzer port 已確認、功率在軟體限制內），且無論流程如何結束都要在 `finally` 關閉 RF。
- 3 絕不自動 **Reset 儀器**（`reset_on_connect: false`），避免清掉既有的 CMsquares Workspace。
- 4 無效量測值不可當成 0（`---`、`NCAP`、`NAN`、`INF` 等一律視為 invalid，保留原始回應）。
- 5 沒有明確設定 **Limit** 就不能宣稱 **Pass**。
- 6 所有 SCPI 存取都要走 `instrument/` + `scpi/` 抽象層，不可在 workflow/CLI/GUI 程式碼裡直接寫 SCPI 字串。

完整規則另見 `SPEC.MD` 第 31、35 節。

開發階段

階段	內容	狀態
Phase 0	Discovery — 實機 SCPI 指令錄製與驗證	未開始
Phase 1	Manual Baseline — 手動操作 SOP 與基線結果	未開始
Phase 2	Connection Framework — 設定/連線骨架、CLI、GUI	進行中 (目前)
Phase 3	Single Measurement — 單次量測 workflow	未開始
Phase 4	Power Sweep — 功率掃描、圖表、Checkpoint	未開始
Phase 5	Frequency Sweep — 頻率掃描、Band 驗證	未開始
Phase 6	文件與 Demo	未開始

目前完成到 Phase 2 的骨架部分；Phase 0 (實機 SCPI Discovery) 仍需要在儀器上手動操作 SCPI Recorder 才能繼續往下。

重要提醒

尚待確認項目 (RF1.1 實際功能、External Attenuation、Remote Interface、所有 WLAN 量測 SCPI 指令等) 不應由開發者或 AI 自行假設或猜測，必須經過實機 Discovery 階段確認後才能寫入設定檔與程式碼。